

Compiling Code LLMs into Lightweight Executables

JIEKE SHI and JUNDA HE, Singapore Management University, Singapore

ZHOU YANG*, University of Alberta & Alberta Machine Intelligence Institute (Amii), Canada

CHENGRAN YANG, Singapore Management University, Singapore

MYKHAILO KLYMENKO, THONG (JAMES) HOANG, SHERRY (XIWEI) XU, and ZHEN-
CHANG XING, CSIRO's Data61, Australia

DAVID LO, Singapore Management University, Singapore

The demand for better prediction accuracy and higher execution performance in neural networks continues to grow. The emergence and success of Large Language Models (LLMs) have produced many cloud-based tools for software engineering tasks such as code suggestion. Although effective, cloud deployment raises concerns over privacy, latency, and reliance on network connectivity. Running LLMs locally on personal devices such as laptops would address these issues, because it enables offline use and reduces response time. However, local deployment is challenging, since commodity devices lack high-performance accelerators such as GPUs and are constrained by limited memory and compute capacity, which makes it hard to execute large models efficiently.

We present DITTO, a framework that optimizes both the model size of Code LLMs and the inference programs that execute them, with a focus on statically-typed languages such as C. Our approach integrates two components. The first is a quantization technique inspired by product quantization, which groups model parameters into per-block codebooks via K-Means clustering and stores each weight as a bit-packed low-bitwidth index. The quantizer further supports a mixed-precision mode that keeps a small number of sensitivity-critical tensors in float32. The second component is a compilation pass integrated into LLVM that automatically detects and replaces unoptimized General Matrix-Vector Multiplication (GEMV) operations, which are the most computationally intensive part of code models, with calls into Basic Linear Algebra Subprograms (BLAS) libraries that are highly optimized for the target hardware. The output of DITTO is a compiled executable that runs the selected Code LLM on commodity hardware.

We evaluate DITTO on three popular Code LLMs, namely Code Llama, MagicCoder, and OpenCodeInterpreter, achieving up to 10.5× faster inference, 6.4× lower memory usage, and 10.5× lower energy consumption compared with their original inference pipelines, while preserving accuracy close to the full-precision models, with an average loss of only 0.27% in pass@1. DITTO also outperforms the state-of-the-art INT8 quantization baseline, achieving up to 6.96% higher pass@1 accuracy, 2.2× speedup, and 1.6× memory reduction, which demonstrates the effectiveness of our approach.

CCS Concepts: • **Software and its engineering** → **Source code generation**; • **Computing methodologies** → **Artificial intelligence**.

Additional Key Words and Phrases: Large Language Models, Compilers, Matrix Multiplication, LLVM

*Corresponding Author, Canada CIFAR AI Chair.

Authors' Contact Information: [Jieke Shi](mailto:jiekeshi@smu.edu.sg), jiekeshi@smu.edu.sg; [Junda He](mailto:jundahe.2022@smu.edu.sg), jundahe.2022@smu.edu.sg, Singapore Management University, Singapore; [Zhou Yang](mailto:zy25@ualberta.ca), University of Alberta & Alberta Machine Intelligence Institute (Amii), Canada, zy25@ualberta.ca; [Chengran Yang](mailto:cryang@smu.edu.sg), Singapore Management University, Singapore, cryang@smu.edu.sg; [Mykhailo Klymenko](mailto:Mykhailo.Klymenko@data61.csiro.au), Mike.Klymenko@data61.csiro.au; [Thong \(James\) Hoang](mailto:james.hoang@data61.csiro.au), james.hoang@data61.csiro.au; [Sherry \(Xiwei\) Xu](mailto:Sherry (Xiwei) Xu, Xiwei.Xu@data61.csiro.au), Xiwei.Xu@data61.csiro.au; [Zhenchang Xing](mailto:Zhenchang.Xing@data61.csiro.au), Zhenchang.Xing@data61.csiro.au, CSIRO's Data61, Australia; [David Lo](mailto:davidlo@smu.edu.sg), Singapore Management University, Singapore, davidlo@smu.edu.sg.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE189

<https://doi.org/10.1145/3808196>

ACM Reference Format:

Jieke Shi, Junda He, Zhou Yang, Chengran Yang, Mykhailo Klymenko, Thong (James) Hoang, Sherry (Xiwei) Xu, Zhenchang Xing, and David Lo. 2026. Compiling Code LLMs into Lightweight Executables. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE189 (July 2026), 25 pages. <https://doi.org/10.1145/3808196>

1 Introduction

In recent years, Large Language Models (LLMs) have achieved remarkable success across various domains. Among them, specialized LLMs trained on large amounts of source code, commonly known as Code LLMs, have demonstrated exceptional accuracy in automating tasks such as code generation [16, 28]. These models have become integral to modern software development, since they enable developers to write code more efficiently and effectively [47, 74]. Their success has led to a variety of tools, such as GitHub Copilot [21] and Cursor [11], that leverage these capabilities to assist developers in everyday tasks [64, 74, 76].

While most Code LLM-based applications are deployed on cloud servers to leverage high-performance computing resources, there are notable benefits to running them on personal devices such as laptops, provided that their memory and computational demands can be reduced to levels manageable by commodity hardware. Recent studies show that developers increasingly prefer local execution of Code LLMs over cloud-based alternatives [35, 44, 54, 56], because local execution improves privacy, lowers latency, and supports offline use. Running Code LLMs locally allows developers to retain control over their code and data, which reduces the risk of exposing sensitive information to third-party services. In addition, local execution removes the dependency on network connectivity, which enables development in environments with limited or no internet access. It also reduces latency by eliminating the data transmission delay to and from cloud servers, a delay that is often affected by network conditions. This matters most for real-time applications such as code completion and debugging, where quick responses are essential. Nonetheless, state-of-the-art Code LLMs typically contain billions of parameters and require 20 to 30 GB of memory to run, which exceeds the capacity of common laptops or desktops with 16 GB of memory, so direct local deployment is impractical without specialized optimization.

To date, only a limited number of studies have investigated how to reduce the memory and computational costs of Code LLMs [1, 55, 57, 66]. These efforts primarily adopt high-level strategies, such as reducing the number or bit width of model parameters, without addressing the underlying implementation of model inference from a program optimization perspective. Without low-level optimization, such approaches cannot fully support high-performance execution of Code LLMs on personal devices. For example, although an increasing number of inference pipelines are implemented in C or C++, such as llama2.c [32], which was developed by one of OpenAI's founders to run the LLaMA-series models efficiently, their performance often remains suboptimal, because of the lack of systematic low-level optimizations and the frequent reliance on manual tuning for specific hardware. In addition, some implementations depend on specialized high-performance computing hardware, such as GPUs or CPUs with high memory bandwidth, which are not available on most personal devices such as business laptops. As a result, their inference programs still require substantial improvement to achieve efficient execution under constrained hardware resources.

In this paper, instead of treating model optimization solely as a parameter reduction task, we propose a model quantization method accompanied by a compilation-time optimization approach, which frames the compression and acceleration of LLMs as a program optimization problem. We present DITTO, a framework for jointly optimizing the model size and the inference program of Code LLMs, with a current focus on C-based implementations such as llama.c [32]. As a lightweight and portable pipeline with 18k GitHub stars, llama.c provides efficient LLaMA-series inference on edge devices and has gained wide adoption, so we use it as the deployment baseline that our

approach builds upon and further optimizes. During inference, an LLM performs computations involving large numbers of high-precision floating-point parameters. Inspired by the idea of product quantization [46], which leverages clustering to reduce the bit width of model parameters, DITTO treats the model parameters as a set of vectors, clusters them, and stores only the cluster centroids. The original parameters are replaced with indices that point to their assigned centroids, which reduces the bit width of the parameters substantially. After quantization, DITTO automatically synthesizes the inference program for the quantized model.

However, simply reducing the bit width of model parameters is not sufficient for efficient execution on personal devices. The inference of Code LLMs involves numerous General Matrix-Vector Multiplication (GEMV) operations, which are computationally intensive and account for over 80% of total execution time [36, 71]. To optimize this, we design an LLVM compilation pass that automatically detects GEMV operations and replaces them with highly optimized implementations from Basic Linear Algebra Subprograms (BLAS) libraries that are specialized for the target hardware. The combination of quantization and compiler-level optimization yields executables that run Code LLMs efficiently under tight memory budgets while also delivering substantial speedups. We evaluate DITTO on a personal device equipped with an Apple M4 CPU, which is commonly found in consumer laptops. The results show that DITTO enables LLM inference up to 10.5× faster than the original inference pipeline on the same device, while reducing memory usage by up to 6.4× and energy consumption by up to 10.5×, and preserving accuracy close to that of the original models, with at most a 4.45% loss in pass@1 and an average loss of only 0.27%. Furthermore, DITTO outperforms an INT8 quantization baseline, achieving up to 6.96% higher pass@1 accuracy, up to 2.2× speedup, 1.6× lower memory usage, and 2.2× energy savings.

Contributions. Our goal in this work is to close the gaps in Code LLM deployment on personal devices that were discussed above. In summary, we make the following contributions:

- We present DITTO, a framework that automatically optimizes the execution of Code LLMs, which enables them to run on personal devices with limited memory and computational resources.
- We propose a quantization scheme paired with a compiler-inspired rewrite, which reduces the bit width of model parameters and accelerates the inference program while preserving accuracy.
- We evaluate DITTO on popular Code LLMs and show that it can run these models on personal devices with up to 10.5× faster inference and 6.4× lower memory usage compared to the original inference programs, while preserving functional correctness with an average loss of only 0.27% in pass@1.

2 Preliminaries

2.1 Code LLMs and Their Inference

Code LLMs are specialized Large Language Models (LLMs) trained on large amounts of source code [16, 25, 64, 72], enabling them to perform a variety of software engineering tasks such as code generation [9, 70] and summarization [2, 37], vulnerability detection [60, 77], and program repair [29, 30, 68]. These models are typically based on the Transformer architecture [63], whose main components are self-attention and feed-forward layers. In the feed-forward layers, one of the most computationally intensive operations is the General Matrix-Vector Multiplication (GEMV), which is used to compute their outputs.

General Matrix-Vector Multiplication (GEMV). GEMV is a fundamental linear algebra operation that has regained prominence as a critical building block in neural network inference. In the context of Code LLMs deployed on personal devices, where user queries are typically processed sequentially, GEMV plays a particularly crucial role. During each step of autoregressive generation, the model

must multiply large weight matrices by single input vectors. This operation, precisely GEMV, requires streaming entire weight matrices through memory for each generated token, which creates significant memory bandwidth pressure. Without careful optimization, inference latency becomes bottlenecked by memory transfers rather than compute. Formally, GEMV is expressed as:

Definition 2.1. Let A be a matrix of dimensions $M \times N$ and x be a vector of dimensions $N \times 1$. Let α and β be any real values. The general matrix-vector product of A and x produces a vector y of dimensions $M \times 1$ such that: $y(i) = \beta \cdot y(i) + \alpha \cdot \sum_{k=1}^N A(i, k) \cdot x(k)$, for $i = 1, \dots, M$.

Although a naïve implementation of GEMV is simple, it can not leverage hardware-specific optimizations and is therefore significantly slow. Listing 1 shows a basic row-major implementation of GEMV written in plain C, which serves as a conceptual baseline.

This naïve implementation performs the correct computation but does not benefit from cache blocking, vectorization, or fused multiply-add (FMA) instructions, which are some common optimization techniques [13]. It therefore serves as a useful pedagogical reference but is not suitable for high-performance inference workloads.

Many optimization efforts have focused on improving GEMV performance, which has produced a number of highly optimized implementations such as the Basic Linear Algebra Subprograms (BLAS) libraries [7]. These libraries provide efficient implementations of GEMV and other linear algebra operations by leveraging low-level optimizations and hardware-specific features to achieve high performance, optimizations that are currently not utilized by Code LLM inference programs. Listing 2 shows the interface for double-precision GEMV in common BLAS libraries, which can be used to accelerate GEMV computations in Code LLMs. The Layout argument specifies whether the result vector y is stored in row-major or column-major order. The trans argument indicates whether the matrix A is transposed, which allows multiplication with vectors in different storage orders without creating copies. The parameters m and n correspond to the matrix dimensions, followed by the scalar factors α and β (lines 5 and 8), the base address and leading dimension of A (lines 6 and 7), and the increment values incx and incy for traversing the input and output vectors (lines 7 and 8). In this paper, we focus on optimizing GEMV computations in Code LLM inference programs by replacing the naïve implementation with calls to such optimized libraries, which can significantly improve performance.

```
1 void gemv_naive(const float* A, const float*
2 x, float* y, int m, int n,
3 float alpha, float beta) {
4   for (int i = 0; i < m; ++i) {
5     float sum = 0.0f;
6     for (int j = 0; j < n; ++j) {
7       sum += A[i * n + j] * x[j];
8     }
9     y[i] = alpha * sum + beta * y[i];
10  }
```

Listing 1. Naïve GEMV implementation.

```
1 void cblas_dgemv(
2   const CBLAS_LAYOUT Layout,
3   const CBLAS_TRANSPOSE trans,
4   const int m, const int n,
5   const double alpha,
6   const double *A, const int lda,
7   const double *x, const int incx,
8   const double beta, double *y, const int incy
9 );
```

Listing 2. CBLAS interface for GEMV.

2.2 Model Compression

Model compression aims to reduce the size of a model, and thereby improve other important metrics such as inference latency, memory usage, and energy consumption. Various techniques have been explored for compressing LLMs, including knowledge distillation [24]. As the pioneering work on compressing Code LLMs, Shi et al. [57] propose Compressor, which uses a genetic algorithm to simplify model architectures and applies knowledge distillation to reduce model size to as little as 3

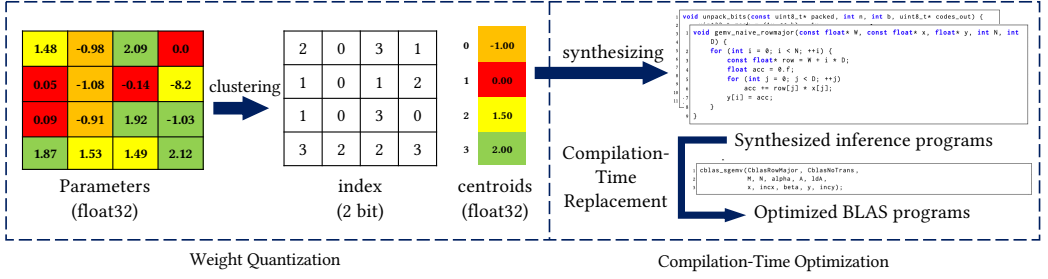


Fig. 1. Overview of DITTO's two-phase optimization framework.

MB. Their compressed models achieve an average $4.23\times$ improvement in inference latency. Building on this, Shi et al. [55] present Avatar, which searches for Pareto-optimal compressed models that jointly optimize model size, inference latency, energy consumption, and carbon footprint, rather than focusing solely on size. Avatar's compressed models reduce energy consumption by up to $184\times$, carbon footprint by up to $157\times$, and inference latency by up to $76\times$. In a similar line of work, Su and McMillan [58] apply knowledge distillation to GPT-3.5, producing a smaller and more efficient model for code summarization.

Quantization is another widely used approach to compression, which reduces the bit width of model parameters. This technique has also been applied to Code LLMs. Wei et al. [66] and Kaushal et al. [34] adopt quantization and low-rank decomposition, respectively, to improve efficiency in code generation. In particular, they directly quantize model parameters to 4-bit integers by truncating the least significant bits of the original floating-point values. Although simple and effective, this approach treats each weight independently and does not adapt to the distribution of values within each layer. In contrast, DITTO adopts a clustering-based quantization strategy that groups model parameters into codebooks and replaces them with low-bitwidth centroid indices, which gives each layer an adaptive representation that follows the actual distribution of its weights.

3 Methodology

3.1 Overview

DITTO is a two-phase framework that combines weight quantization with program-level optimization to accelerate inference for Code LLMs. Figure 1 gives an overview of the workflow. The first phase compresses model weights and synthesizes a matching inference program, whereas the second phase optimizes the execution of that program on the target hardware.

In the first phase, DITTO applies a K-Means-based quantization scheme that groups weights into a small number of representative values (centroids) and stores each weight as the index of its assigned centroid. The quantizer operates at the sub-block level, which means that each row of a weight matrix is divided into sub-blocks of G consecutive elements, and each sub-block receives its own codebook. This layout adapts the representation to local weight distributions, which is especially beneficial for large models whose attention and projection matrices contain outlier channels. The quantizer also supports a mixed-precision mode that keeps a small number of sensitivity-critical tensors in float32, which lets the user trade a moderate increase in file size for higher fidelity on specific tensors. Centroid indices are serialized with a bit-packing layout that matches the bit width of the indices, so the stored file size is close to the theoretical lower bound. In the second phase, DITTO applies a custom LLVM IR pass that identifies GEMV loops in the synthesized inference program and rewrites them into calls to a hardware-tuned BLAS routine such as `cblas_sgemv`. The

two phases are complementary, since quantization reduces the memory traffic required by each matmul, whereas the compilation pass accelerates the matmul itself.

3.2 Weight Quantization

3.2.1 K-Means Clustering with Magnitude Clipping. The first aim of DITTO is to represent model weights with a small set of representative float32 values, which amounts to clustering the weights and assigning one representative per cluster. We use 1-D K-Means, because the weights of each layer are scalars and K-Means admits a much faster implementation when its input is one-dimensional. The general K-Means procedure, often referred to as *Lloyd's algorithm* [42], alternates between two steps, namely the *assignment* step that maps each data point to its nearest centroid, and the *update* step that moves each centroid to the mean of its assigned points. For d -dimensional data with n points and k clusters, the assignment step costs $O(nkd)$ per iteration, because every point is compared against every centroid. In one dimension, however, the optimal clusters are always contiguous intervals in the sorted data. A single sort, done once up front, is therefore sufficient, and each subsequent iteration only needs to update the k interval boundaries rather than revisit every point. This reduces the per-iteration cost to $O(k \log n)$ and makes the algorithm well suited for the hundreds of thousands of small clustering problems that our block-wise quantizer generates.

DITTO starts by placing the sorted weights into $k = 2^b$ bins of equal population, where b is the target bit width. This is equivalent to a quantile initialization, which naturally allocates more centroids where the weight density is higher. The centroid of each bin is then refined by Lloyd's iteration. Two subtle but important choices drive the quality of the result:

- **Efficient centroid update via prefix sums.** After sorting the weights once, we maintain a prefix-sum table so that the mean of any contiguous interval can be computed in constant time. Each iteration locates the new cluster boundaries by binary-searching the midpoints of adjacent centroids within the sorted weight sequence, which keeps the per-iteration cost linear in the number of clusters rather than in the number of weights.
- **Magnitude-based scale search.** Let $a = \max_i |w_i|$ denote the absolute maximum of the block, which we refer to as the *absmax*. A direct K-Means fit on the raw weights is easily skewed by a handful of outliers near $\pm a$, since placing centroids to cover them diverts budget away from the bulk of the distribution and inflates the overall reconstruction error. To counter this, we introduce a scale factor f that controls how aggressively the tails are suppressed during fitting. For each candidate f in a predefined grid, we clip the weights to $[-fa, fa]$ and fit K-Means on the clipped sequence, but perform the assignment step on the *original* unclipped weights, which sends any outlier to the extremal centroid. The value of f that minimizes the reconstruction MSE on the original weights is kept. This decoupling of fitting from assignment frees centroid budget from a few outliers and reallocates it to the bulk of the distribution, which is especially helpful at low bit widths. We use $f \in \{0.5, 0.65, 0.8, 0.9, 0.95, 1.0\}$ as the default grid: it is denser near 1.0, since the optimum for most rows lies close to the *absmax* scale, and sparser toward 0.5 to cover rows with heavier tails. The same six-point grid is used in established low-bit quantizers such as GGML's *k-quants* [20], and we find that it suffices for every layer we quantize.

We also apply an early-exit heuristic. Let $\mu = \frac{1}{n} \sum_i |w_i|$ denote the mean absolute value of the block. When $a \leq 3.5\mu$, the block is Gaussian-like, no outliers dominate, and $f = 1.0$ is already optimal. In this case, the entire grid search is skipped. Empirically, this heuristic fires on the majority of blocks in well-behaved layers, which makes the scheme almost free in the common case. Algorithm 1 shows the procedure for a single weight block.

Example. Consider a weight block of nine values,

$$w = [0.91, 0.92, 0.89, -0.05, -0.06, -0.04, 1.20, 1.21, 1.19],$$

and suppose we want to quantize it into three clusters. DITTO first sorts the values,

$$s = [-0.06, -0.05, -0.04, 0.89, 0.91, 0.92, 1.19, 1.20, 1.21],$$

partitions them into three equal-population clusters, and averages each cluster to obtain the codebook $c = [-0.05, 0.91, 1.20]$. Each weight is then replaced with the index of its nearest centroid, which yields the sequence $q = [1, 1, 1, 0, 0, 0, 2, 2, 2]$. Since there are only three centroids, each index can be stored in $\lceil \log_2 3 \rceil = 2$ bits. Compared with the original float32 weights, the quantized indices occupy 16× less memory ($32 \rightarrow 2$ bits per element). The codebook itself occupies only $3 \times 32 = 96$ bits and is shared across all weights in the block, so its overhead is negligible whenever the block is large.

3.2.2 Block-wise Granularity and Mixed Precision.

Clustering can be applied at several granularities, and the choice directly affects the trade-off between storage overhead and quantization fidelity. The coarsest option is to share a single codebook of 2^b centroids across an entire row of the weight matrix. This minimizes codebook storage but is fragile: if any column of that row carries outlier values, those outliers consume a disproportionate share of the 2^b centroids, leaving the remaining weights poorly represented. To contain the damage, DITTO instead partitions each row into sub-blocks of G consecutive elements and assigns every sub-block its own codebook. An outlier now only pollutes the centroid budget of its local G -element block, so the remaining $(n - G)$ weights in the row are unaffected. The price is a modest increase in storage, namely $(n/G) \cdot 2^b$ float32 centroids per row in addition to the

bit-packed indices. The user selects G at quantization time, and we find that $G \in \{128, 256\}$ strikes a good balance between codebook overhead and adaptation to local weight distributions. Our quantizer processes all sub-blocks uniformly: the $r \times c$ weight matrix is reshaped into $(r \cdot c/G) \times G$, and the same Lloyd's kernel is invoked independently on each row of the reshaped matrix.

Mixed Precision. Some tensors, such as the Q and K projections in the attention block, are highly sensitive to quantization error, because they feed into a softmax whose gradient is large near outliers. In these cases, low-bit quantization can still degrade downstream quality, even at block-wise granularity. To address this, DITTO supports a per-tensor-type *fp32 mask*. When the mask flags a tensor type, the quantizer stores that tensor as raw float32 and bypasses clustering entirely. The resulting file uses a mixed layout, where most tensors are quantized and the flagged ones are stored in full precision. This pairs naturally with the compilation-time GEMV pass (Section 3.4),

Input: weights $w \in \mathbb{R}^n$, bit width b , grid $\mathcal{F}$

Output: centroids $c \in \mathbb{R}^{2^b}$, indices $q \in \{0, \dots, 2^b - 1\}^n$

```

1  $s \leftarrow \text{sort}(w)$ ;
2  $a \leftarrow \max_i |w_i|$ ;
3  $\mu \leftarrow \frac{1}{n} \sum_i |w_i|$ 
4 if  $a \leq 3.5 \mu$  then
5    $\mathcal{F} \leftarrow \{1.0\}$            // Gaussian-like: skip grid
6 end
7  $\text{best\_mse} \leftarrow \infty$ 
8 foreach  $f \in \mathcal{F}$  do
9    $s' \leftarrow \text{clip}(s, -fa, fa)$ 
10  Initialize  $c$  by quantile of  $s'$ 
11  repeat
12    Update cluster boundaries by binary-searching
      midpoints of  $c$  in  $s'$ 
13    Update centroids via prefix-sum means over  $s'$ 
14  until convergence
15  Assign each  $w_i$  to its nearest centroid  $c_{q_i}$  and compute
    MSE on  $w$ 
16  if  $\text{MSE} < \text{best\_mse}$  then record  $(c, q, f)$ 
17 end
18 return  $(c, q)$ 
```

Pseudocode 1: DITTO's Weight Quantization Workflow

```

1 void pack_bits(const uint8_t* codes,
2               int n, int b, uint8_t* out) {
3     uint32_t mask = (1u << b) - 1u;
4     int bit_off = 0;
5     for (int i = 0; i < n; ++i) {
6         uint32_t v = codes[i] & mask;
7         int j = bit_off >> 3;
8         int sh = bit_off & 7;
9         out[j] |= uint8_t((v << sh) & 0xFF);
10        if (sh + b > 8)
11            out[j+1] |= uint8_t(
12                (v >> (8 - sh)) & 0xFF);
13        bit_off += b;
14    }
15 }

```

Listing 3. Bit-packing b -bit codes into bytes.

```

1 void unpack_bits(const uint8_t* packed,
2                 int n, int b, uint8_t* codes_out) {
3     uint32_t mask = (1u << b) - 1u;
4     int bit_off = 0;
5     for (int i = 0; i < n; ++i) {
6         int j = bit_off >> 3;
7         int sh = bit_off & 7;
8         uint32_t chunk =
9             uint32_t(packed[j]) |
10            (uint32_t(packed[j+1]) << 8);
11        codes_out[i] = uint8_t(
12            (chunk >> sh) & mask);
13        bit_off += b;
14    }
15 }

```

Listing 4. Bit-unpacking b -bit codes via a 16-bit sliding window.

since fp32 tensors traverse the naive matmul path and are rewritten into cblas_sgemv calls by the pass, which means that keeping a tensor in fp32 does not preclude a BLAS-accelerated execution.

Parallel Implementation. DITTO’s quantizer is written in Python and uses Numba’s just-in-time (JIT) compiler with prange to parallelize Lloyd’s iteration across all sub-blocks of a weight matrix. Each block is sorted once, its prefix-sum representation is reused across all candidates in the scale-factor grid, and the scratch arrays for boundaries and centroids are kept small and thread-local. For a 7B-parameter model, this lets DITTO complete quantization of all seven weight types across 32 layers in about two minutes on a commodity laptop, without relying on multiprocessing.

3.2.3 Bit-Packing. The output of clustering is a sequence of b -bit indices. To realize the memory savings from low-bit representation, these indices must be packed into a dense byte stream, since standard data types are byte-aligned and storing a sub-byte index in an 8-bit container would waste a significant fraction of the storage, for example 50% for a 4-bit index. DITTO therefore implements a low-level bit-packer whose format matches the layout expected by the decoder at inference time. Listing 3 shows the packing routine, and Listing 4 shows the decoder.

Packing. The pack_bits routine serializes a sequence of b -bit indices into a contiguous byte array. A running counter bit_off tracks the next available bit position in the output. For each index v , we need to locate which byte of the output it belongs to and where inside that byte it starts. Since each byte holds $8 = 2^3$ bits, the target byte index is bit_off/8 and the in-byte shift is bit_off mod 8. These are computed cheaply as $j = \text{bit_off} \gg 3$ and $\text{sh} = \text{bit_off} \& 7$, using the bit-level identities that divide-by- 2^3 equals right-shift by 3 and mod- 2^3 equals a bitwise AND with $7 = 2^3 - 1$. The value v is then shifted left by sh and merged into out[j] using a bitwise OR, which preserves the bits that were already packed. When an index crosses a byte boundary, which is detected by the condition $\text{sh} + b > 8$, the overflow bits are shifted right and merged into out[j+1]. The counter bit_off is then incremented by b .

Unpacking. unpack_bits performs the inverse operation. The primary challenge is to read b bits that may begin at any bit offset. The decoder uses a small *sliding window*, formed by combining packed[j] and packed[j+1] into a 16-bit chunk, shifting the chunk right by sh, and masking out the b least-significant bits. This avoids branching on the byte-boundary case and keeps the decoder branch-free. To keep the window safe at the tail of the stream, DITTO appends a single guard byte to every packed tensor, so the read of packed[j+1] always falls within the allocated buffer.

For bit widths whose packed layout aligns cleanly with byte boundaries, the generic loop is specialized into a fast path that avoids the per-index shift and mask arithmetic. The relevant condition is that a small number of b -bit indices fits into an integer number of bytes, which happens whenever $b \cdot k$ is a multiple of 8 for some small k . Three bit widths in particular satisfy this: with $b = 8$, each byte is a full index ($k = 1$); with $b = 4$, two indices fill one byte ($k = 2$); and with $b = 3$, eight indices fill three bytes exactly ($k = 8$, since $3 \cdot 8 = 24$ bits), so the decoder can load three bytes into a 24-bit accumulator and extract all eight indices in a straight-line sequence of shifts. These fast paths are hand-written in `run_ditto.c` and together cover every bit width that DITTO uses in practice, whereas any other bit width falls back to the generic 16-bit sliding window.

3.3 Inference Kernel

Quantization changes what is on disk; it also changes what the inference kernel must do on every forward pass. Instead of reading a row of float32 weights directly into a dot product, the kernel must first unpack the b -bit indices, look them up in a centroid table, and only then compute the dot product with the activation vector. DITTO ships a hand-tuned quantized GEMV kernel for this purpose, and a small dispatcher that routes each matmul call to either the quantized kernel or to the naive `matmul()` function that the LLVM pass replaces with `cblas_sgemv`.

Quantized GEMV. The quantized kernel, shown schematically in Listing 5, iterates over the output rows in parallel using OpenMP. Each iteration unpacks the packed indices of its row into a thread-local float32 scratch buffer, looks the indices up in the corresponding sub-block codebook, and computes a dot product between the reconstructed row and the activation vector.

Two design choices are critical to performance. First, the scratch buffer is allocated once per model as a single contiguous region of size $T \cdot D_{\max} \cdot 4$ bytes, where T is the maximum number of OpenMP threads and $D_{\max}$ is the largest row length that the model uses. Each thread takes a disjoint slab of this region, indexed by `omp_get_thread_num()`, which eliminates the per-iteration `malloc/free` that a naive implementation would require. A 7B-parameter model makes roughly 1.4 million GEMV row calls per generated token, so avoiding heap traffic in this loop yields a measurable throughput improvement.

Second, `unpack_row` dispatches into a fast path for $b \in \{3, 4, 8\}$ and into a generic 16-bit-sliding-window decoder for all other bit widths. The fast paths exploit the exact byte layout of their bit width, so they avoid shift arithmetic and branch decisions in the hot loop. The inner `dot_product` call is hand-vectorized with ARM NEON intrinsics on Apple Silicon and with SSE/AVX on x86, using four independent accumulators to hide the latency of fused multiply-add (FMA) instructions.

Dispatcher. The forward pass does not call `ditto_matmul` directly. Instead, it calls a small wrapper, `tensor_matmul`, that inspects the `is_fp32` flag of the tensor and dispatches either to `ditto_matmul` (for quantized tensors) or to the naive `matmul` function (for fp32 tensors flagged by the mixed-precision mask). The fp32 path is written as a plain doubly-nested loop with no SIMD intrinsics, which is exactly the pattern that the LLVM pass in Section 3.4 is designed to rewrite. As a result, mixed-precision tensors benefit from both the fidelity of fp32 storage and the speed of BLAS, without any hand-written code in `run_ditto.c`.

```

1 void ditto_matmul(
2     float* xout, const float* x,
3     const QuantTensor* qt,
4     int n, int d, int bits, int k,
5     int G, float* scratch,
6     int row_stride) {
7     #pragma omp parallel for
8     for (int i = 0; i < d; ++i) {
9         int tid = omp_get_thread_num();
10        float* buf = scratch +
11            (size_t)tid * row_stride;
12        unpack_row(qt, i, n, bits, k, G, buf);
13        xout[i] = dot_product(buf, x, n);
14    }
15 }
```

Listing 5. Quantized GEMV with per-thread scratch.

3.4 Compilation-Time Optimization

The second phase of DITTO is a custom LLVM IR pass that rewrites naive GEMV loops into calls to a tuned BLAS routine. This phase complements the quantization phase, which targets memory traffic, by targeting compute efficiency. The pass is implemented on top of LLVM's PatternMatch API and is loaded into the standard optimization pipeline as a plugin. The pipeline runs in three stages, namely pattern identification, legality and dependency checking, and library replacement.

Stage 1: Pattern Identification. A GEMV pattern in DITTO is a loop nest of depth at least two, whose innermost body performs a multiply-add reduction over affine memory accesses. Identifying this pattern in LLVM IR is non-trivial, since the same source-level loop can lower into many different IR shapes depending on the optimization level, the target architecture, and the way the induction variable is represented. DITTO therefore extends LLVM's built-in matchers with three custom combinators. `OneOf` matches any of a set of alternative sub-patterns, which covers the common variations in affine index computation. `PHITimesLD` matches the multiplication between a loop induction variable and a leading dimension, which is the canonical form of a row stride. `AffineFunctionOfPHI` matches more general affine expressions of one or more loop induction variables. These combinators are composed into a higher-level `GEMVReduction` matcher that looks for the multiply-add reduction itself, and into `MatchStoreOfVector`, which recognizes the write to the output vector in its many possible forms. Listings 6 to 8 show the matcher code.

```

1 template <typename MatcherType>
2 auto MatchStoreOfVector(MatcherType GEMVReduction,
3                          Value y, Value Alpha, Value Beta,
4                          PHINode iv_i, PHINode iv_k,
5                          Value LDy, Value GEP) {
6     return m_Store(
7         OneOf(ScaledVOrV(Alpha, GEMVReduction),
8               ScaledVOrV(Beta, m_PHI(m_Value()), GEMVReduction),
9               m_c_FAdd(ScaledVOrV(Beta, m_Load(GEP)),
10                      ScaledVOrV(Alpha, m_PHI(m_Value()), GEMVReduction))),
11         ArrayAccess(y, iv_i, iv_k, LDy));
12 }

```

Listing 6. Matcher for storing a GEMV result into the output vector.

```

1 inline auto GEMVReduction(
2     Value AddLHS, Value MulLHS,
3     Value MulRHS, PHINode iv_i,
4     PHINode iv_k, Value Alpha,
5     Value LDA) {
6     return MultiplyAdd(
7         Alpha, AddLHS,
8         m_Load(ArrayAccess(
9             MulLHS, iv_i, iv_k, LDA)),
10        m_Load(ArrayAccess(
11            MulRHS, iv_k)));
12 }

```

Listing 7. Matcher for the multiply-add reduction in the loop body.

```

1 auto ArrayAccess(
2     Value Op, PHINode iv1,
3     PHINode iv2, Value LD) {
4     return OneOf(
5         m_GEP(m_Load(m_GEP(Op, m_PHI(iv1))),
6               m_PHI(iv2)),
7         m_GEP(m_GEP(Op, PHITimesLD(iv1, LD)),
8               m_PHI(iv2)),
9         m_GEP(Op, m_PHI(iv1), m_PHI(iv2)),
10        m_GEP(Op, AffineFunctionOfPHI(
11            iv1, iv2, LD)));
12 }

```

Listing 8. Matcher for array access variants.

DITTO's IR pass then scans each function for candidate GEMVs using these matchers, as outlined in Algorithm 2. The pass walks every loop in the function's `LoopInfo`, selects the ones whose depth is at least two, and applies the matchers to every instruction in the innermost body. When a match succeeds, the pass records the candidate together with the pointers, induction variables, and leading dimension that the matcher has already bound.

Stage 2: Legality and Dependency Checks. Once a candidate is matched, it is subject to a legality check that ensures the rewrite is sound. The core of the check is a data-dependence analysis on the candidate's inner loop, which rejects any loop that contains side effects other than the accumulator update, such as additional stores, aliasing loads, or calls with memory effects. Candidates that fail this check are dropped.

DIRTTO also infers the memory layout of the matrix by inspecting the address computation of the matrix load. If the offset is of the form $i \cdot ld_A + k$, the layout is row-major. If it is $k \cdot ld_A + i$, the layout is column-major. Any other pattern causes the candidate to be dropped. In practice, this conservative design produced no compilation failures or silent rewrite errors across all evaluated models, and 100% of the GEMV operations in the synthesized inference programs matched our canonical pattern and were rewritten successfully.

Stage 3: Library Replacement. The final stage emits a call to a tuned BLAS routine, such as `cblas_sgemv`. The pass inserts the call at the preheader of the candidate's outer loop, using the pointers, dimensions, and leading dimension that were bound in Stage 1. It then removes the store to `y[i]` in the original inner loop. After the pass finishes, the standard LLVM cleanup passes, namely Dead Code Elimination and CFG simplification, remove the now-unused loop nest. The BLAS symbol itself is declared at IR level using `Module::getOrInsertFunction`, so DIRTTO's generated IR does not require `run_ditto.c` to include `<cblas.h>`. At link time, the symbol is resolved against the system BLAS, which is Apple Accelerate on macOS and OpenBLAS or Intel MKL on Linux. Listing 9 shows a matched GEMV idiom, and Listing 10 shows the rewritten code.

```
1 for (int i = 0; i < M; ++i) {
2   float sum = 0.0f;
3   for (int j = 0; j < N; ++j)
4     sum += A[i][j] * x[j];
5   y[i] = sum;
6 }
```

Listing 9. Matched GEMV idiom in C.

Input: function F , loop info LI
Output: list of GEMV candidates

```
1 foreach loop in LI do
2   if loop depth ≥ 2 then
3     Find innermost sub-loops;
4     foreach value  $V$  in innermost body do
5       if GEMVPattern.Match( $V$ ) then
6         Record candidate;
```

Pseudocode 2: LLVM IR pass for locating GEMV candidates.

```
1 cblas_sgemv(CblasRowMajor,
2   CblasNoTrans,
3   M, N, alpha, A, ldA,
4   x, incx, beta, y, incy);
```

Listing 10. Rewritten CBLAS call.

3.5 Implementation

DIRTTO is implemented in approximately 4,000 lines of Python and C++, including tests and documentation. The quantizer is written in Python with Numba, the inference kernel and the forward pass are written in C on top of llama2.c [32], and the LLVM pass is written in C++ using the new pass manager API. The build pipeline is expressed as a three-stage Makefile target. Stage 1 compiles `run_ditto.c` to LLVM IR at `-O1`, which preserves the loop structure of the naive matmul so that Stage 2 can recognize it. Stage 2 runs `opt` with the DIRTTO pass plugin and rewrites GEMV loops into `cblas_sgemv` calls. Stage 3 compiles the rewritten IR to a native executable at `-O3` with `-ffast-math` and links against the system BLAS. Model weights and tokenizers are converted from the Hugging Face checkpoint format into llama2.c's native binary layout via an export pipeline, after which DIRTTO operates on the converted binaries. We use $b=4$ and $G=256$ as the default quantization

configuration, which empirically provides the best trade-off among accuracy, memory footprint, throughput, and energy consumption.

4 Evaluation

4.1 Experimental Setup

Platform. We evaluate DITTO on a machine with an Apple M4 Chips, 24GB of RAM. We use Clang 14 as the compiler to compile the generated C code into an executable binary. The BLAS library used in our experiments is Apple’s Accelerate [3], which is optimized for Apple Silicon chips. The operating system that we use is macOS 13.2.1.

Comparisons. We evaluate DITTO by comparing the performance of the optimized programs it generates against two existing implementations as baselines. Note that in the following experiments, we set DITTO to use its default 4-bit per-block configuration ($b=4$, $G=256$) for all models. We leave the exploration of different bit widths to future work.

We first compare DITTO against the original floating-point implementation in llama2.c [32], which has been used in previous work [51, 66]. We chose llama2.c as the baseline for several reasons. First, llama2.c, which was developed by OpenAI co-founder Andrej Karpathy, represents the state of the art in LLM deployment on edge devices and features highly optimized kernel implementations tailored to each hardware platform. Its versatility and robust performance make it an ideal baseline. Second, llama2.c is implemented in plain C and C++ without external dependencies, which ensures maximum compatibility and efficiency across diverse hardware configurations. We use the llama2.c implementation at commit d0f8b3c.

Moreover, we also compare DITTO against a strong INT8 baseline [66], a widely adopted quantization method that has demonstrated competitive performance in code generation tasks in prior studies [1, 17, 66]. In our experiments, this baseline is implemented using PyTorch’s native post-training dynamic quantization, following the setup of Wei et al. [66] from AWS AI Lab. Concretely, in our INT8 baseline, all linear layers are quantized from float32 weights into int8 representations, while activations remain in floating-point format. The quantized model is executed with PyTorch’s optimized backend, which leverages efficient low-level libraries to accelerate inference on modern CPU architectures. The INT8 baseline is implemented with PyTorch 2.8.0 with CPU backend.

Models. We evaluate DITTO on three popular code-oriented LLMs: Code Llama [53], Magicoder [67], and OpenCodeInterpreter [75], all of which are open-source and available on Hugging Face [14] and have demonstrated strong performance on code generation tasks across various benchmarks [40, 78].

Code Llama [53] is a code-specialized variant of Llama 2 [61] from Meta, created by further training Llama 2 on code-specific datasets. It can generate both code and natural language descriptions of code, given prompts in either form, and supports multiple programming languages. Magicoder [67] is another code-specialized model built on top of Code Llama, further trained on a large corpus of clean and synthetic data, and demonstrates improved performance in code generation. OpenCodeInterpreter [75] is a more recent advancement that integrates both code execution and human feedback into its training data, achieving performance comparable to proprietary models such as the GPT-4 series. We use the 7B parameter version of each model, as this size is both widely adopted in practice and represents the upper bound for running full-precision models on a typical laptop with 16 GB or 24 GB of memory. In the following sections, we refer to these models as Code Llama-7B, Magicoder-CL-7B, and OpenCodeInterpreter-CL-7B, following the naming conventions in their official Hugging Face repositories. The 7B versions consist of 32 layers, a hidden size of 4,096, and a feed-forward network dimension of 11,008, with a total size of 28 GB in 32-bit floating point. We use 4-bit per-block quantization for all three models, which is the default setting in DITTO.

Table 1. Performance comparison of full-precision, DITTO, and our baseline which runs in INT8 format for different models on HumanEval+ and MBPP+ datasets.

Model		HumanEval+		MBPP+	
		Pass@1	Pass@5	Pass@1	Pass@5
Code Llama-7B [53]	Full-precision	35.41	52.69	46.80	61.20
	DITTO	36.66 (+1.25)	50.14 (-2.55)	47.29 (+0.49)	56.75 (-4.45)
	INT8	29.70 (-5.71)	44.57 (-8.12)	42.41 (-4.39)	48.95 (-12.25)
Magicoder-CL-7B [67]	Full-precision	60.45	78.55	64.20	73.29
	DITTO	60.68 (+0.23)	76.27 (-2.28)	62.50 (-1.70)	73.54 (+0.25)
	INT8	55.84 (-4.61)	70.97 (-7.58)	57.10 (-7.10)	66.64 (-6.65)
OpenCodeInterpreter-CL-7B [75]	Full-precision	67.70	78.44	64.20	71.84
	DITTO	65.87 (-1.83)	74.15 (-4.29)	62.83 (-1.37)	68.79 (-3.05)
	INT8	65.30 (-2.40)	76.54 (-1.90)	59.88 (-4.32)	67.64 (-4.20)

Benchmarks. We evaluate DITTO on two widely used benchmarks for code LLMs: HumanEval+ and MBPP+. HumanEval [9] and MBPP [5] are among the most commonly used benchmarks for evaluating code generation. Each task in these benchmarks includes a prompt in the form of a task description (e.g., a docstring), and the LLM is expected to generate code that is evaluated against a set of test cases to verify correctness. The original HumanEval benchmark consists of 164 Python programming problems, whereas MBPP includes 974 problems. For a more comprehensive evaluation, we use their extended versions, HumanEval+ and MBPP+, which are enhanced by EvalPlus+ [40] with additional test cases, namely 80 times more for HumanEval and 35 times more for MBPP. We use the official evaluation scripts provided by the benchmarks to assess model performance. The evaluation metrics include pass@1 and pass@5, which measure the percentage of problems for which the generated code passes all test cases on the first and fifth attempts, respectively. We also measure memory usage and inference speed, as these are important non-functional properties for deploying code LLMs on resource-constrained devices.

Research Questions. We aim to answer the following research questions:

- **RQ1:** Can DITTO optimize Code LLMs into executables that produce accurate results?
- **RQ2:** How does DITTO improve runtime efficiency compared to the original implementation?
- **RQ3:** What are the individual contributions of weight quantization and compilation-time GEMV optimization to DITTO's overall performance?

4.2 RQ1: Functionality Preservation

Preservation relative to full-precision. The results in Table 1 show that DITTO preserves the core code-generation capabilities of state-of-the-art LLMs after 4-bit per-block quantization, with only minor performance degradation in most cases. For pass@1, the preservation is nearly lossless, with an average degradation of just 0.27% across all models, all values within 2%, and the largest drop being 1.83% for OpenCodeInterpreter-CL-7B on HumanEval+. In some cases, DITTO even improves single-solution accuracy, for example +1.25% for Code Llama-7B and +0.23% for Magicoder-CL-7B. These gains align with prior studies [12, 19, 66], which suggest that quantization can act as a mild form of regularization that reduces overfitting to suboptimal completions. For pass@5, the effect is more noticeable, with reductions up to 4.45% (Code Llama-7B on MBPP+). This suggests that, although the quality of the best solution is largely preserved, quantization narrows the diversity of correct solutions. Nevertheless, the degradation remains low, typically under 3%.

Comparison to the INT8 baseline. As Table 1 shows, DITTO also achieves consistent and substantial improvements over the widely used post-training INT8 quantization baseline. On HumanEval+,

Table 2. Memory, inference speed, computational efficiency, and energy comparison on Apple M4 CPU.

Metric	Method	Code Llama-7B	MagicCoder-CL-7B	OpenCodeInterp.-CL-7B
Memory (GB)	Full-precision	26.77	26.14	26.77
	DITTO	4.2 (6.4×)	4.1 (6.4×)	4.2 (6.4×)
	int8	6.7 (4.0×)	6.7 (3.9×)	6.7 (4.0×)
Tokens/s	Full-precision	0.5	0.5	0.4
	DITTO	3.5 (7.0×)	4.7 (9.4×)	4.2 (10.5×)
	int8	2.2 (4.4×)	2.1 (4.2×)	2.2 (5.5×)
Latency (ms/tok)	Full-precision	2000	2000	2500
	DITTO	286 (7.0×)	213 (9.4×)	238 (10.5×)
	int8	455 (4.4×)	476 (4.2×)	455 (5.5×)
Eff. GFLOP/s	Full-precision	6.5	6.5	5.2
	DITTO	45.3 (7.0×)	60.9 (9.4×)	54.4 (10.5×)
	int8	28.5 (4.4×)	27.2 (4.2×)	28.5 (5.5×)
Energy (J/tok)	Full-precision	36.0	36.0	45.0
	DITTO	5.1 (7.1×)	3.8 (9.5×)	4.3 (10.5×)
	int8	8.2 (4.4×)	8.6 (4.2×)	8.2 (5.5×)

DITTO delivers up to +6.96% higher pass@1 and +5.57% higher pass@5 on Code Llama-7B, whereas on MBPP+, the gains reach up to +5.40% in pass@1 on Magicoder-CL-7B and +7.80% in pass@5 on Code Llama-7B. The only exception is pass@5 for OpenCodeInterpreter-CL-7B on HumanEval+, where INT8 slightly outperforms DITTO by 2.39%. Overall, DITTO outperforms the INT8 baseline across all models, with average gains of +4.27% in pass@1 and +4.06% in pass@5 on each benchmark.

To reduce randomness in code generation, each evaluation is run five times with different random seeds, and we report the average results. We also apply the Mann-Whitney U test [45], a non-parametric statistical test for comparing two independent groups, to measure statistical significance. The results show that DITTO significantly outperforms INT8 in all but one case, with a 99% confidence level (p-value < 0.01). These findings demonstrate that DITTO is more effective than INT8 at preserving both correctness and solution diversity across all tested models.

Answer to RQ1: DITTO effectively optimizes state-of-the-art Code LLMs into low-bit programs while preserving their core functionality. The process has a minimal impact on top-1 accuracy (at most a 1.83% drop in pass@1, 4.45% in pass@5), and it consistently outperforms the INT8 baseline by up to +6.96% in pass@1 and +7.80% in pass@5.

4.3 RQ2: Memory Usage, Latency, and Efficiency

Table 2 reports memory footprint, throughput, latency, effective computational efficiency, and energy consumption on Apple M4 CPU. Across all three evaluated models, DITTO consistently outperforms both full-precision inference and the widely adopted INT8 baseline.

Memory Usage. We measure peak memory usage during inference by generating 1,024 tokens from a fixed prompt and averaging over five runs. DITTO reduces the memory footprint from approximately 26 GB to 4.1 to 4.2 GB for 7B-scale models, which corresponds to a uniform 6.4× reduction. In comparison, INT8 reduces memory to around 6.7 GB (3.9 to 4.0×). The additional savings of up to 1.6× over INT8 come from DITTO’s 4-bit per-block representation. Theoretically, 4-bit quantization provides a compression factor of 2× over int8 and 8× over FP32. In practice,

alignment constraints, per-block codebook overhead, and auxiliary data structures such as unpacking buffers moderate these gains, which yields an effective 6.4× reduction. Even so, this reduction enables 7B-scale models to fit within an 8 GB memory budget, which improves the feasibility of commodity-device deployment.

Throughput and Latency. Using the same experimental setup, we measure throughput in tokens per second (tok/s). DITTO achieves 3.5 to 4.7 tok/s across the three models, which represents up to a 10.5× speedup over full precision. The INT8 baseline reaches only 2.1 to 2.2 tok/s (up to 5.5× speedup). In terms of latency, this corresponds to 213 to 286 ms per token for DITTO, which is up to a 10.5× reduction compared to full precision (2000 to 2500 ms) and up to a 2.2× reduction relative to INT8 (455 to 476 ms). These improvements stem from two complementary factors, namely (1) reduced memory traffic because of the smaller 4-bit per-block weight footprint and (2) improved kernel efficiency, since our optimized GEMV integrates low-bit unpacking with BLAS-based vectorized computation that amortizes decoding overhead while preserving high parallelism.

Computational Efficiency. GFLOPs denote the number of floating-point operations required per token and serve as a hardware-agnostic measure of computational workload. Under the same hardware conditions, a higher GFLOP/s indicates more efficient inference and better hardware utilization. To quantify GFLOPs, we first estimate the per-token cost using a tool provided by Google [10], which reports that a 7B-parameter transformer requires approximately 12.95 GFLOPs per token during inference. This estimate includes all operations involved in token generation, such as matrix multiplications and activations. Importantly, this cost is invariant to quantization, because regardless of whether weights are stored in full precision or in a compressed format, the number of arithmetic operations in GEMV remains unchanged. Therefore, although quantization reduces memory usage and may improve data-transfer efficiency, it does not alter the fundamental computational workload per token. Based on this constant, we compute effective throughput as $\text{GFLOP/s} = 12.95 \times \text{tok/s}$. DITTO achieves 45.3 to 60.9 GFLOP/s, which represents up to a 10.5× improvement over full precision (5.2 to 6.5 GFLOP/s) and up to a 2.2× improvement over INT8 (27.2 to 28.5 GFLOP/s), reflecting improved utilization of compute resources.

Energy Efficiency. We estimate energy consumption per token as $E = P/\text{tok/s}$, where P denotes the sustained system-level power consumption during inference (measured in watts, W). We measure P using the powermetrics utility on macOS, which samples CPU package power at 1-second intervals throughout a 1,024-token generation run. Across all configurations, namely full precision, INT8, and DITTO, the observed power draw remains nearly constant at 18 W, which is expected on the Apple M4. As an integrated SoC, the CPU cores remain fully utilized during inference regardless of weight precision. We therefore use an average power of 18 W for all configurations. Under this model, DITTO consumes 3.8 to 5.1 J/tok (joules per token), where J denotes energy in joules. This represents up to a 10.5× reduction relative to full precision (36.0 to 45.0 J/tok) and up to a 2.2× reduction compared with INT8 (8.2 to 8.6 J/tok). These results demonstrate substantial improvements in energy efficiency across all evaluated models.

Answer to RQ2: DITTO reduces memory usage by up to 6.4× and accelerates inference by up to 10.5× relative to full-precision models, while also achieving up to 1.6× memory savings and 2.2× throughput improvements over the INT8 baseline. It further improves effective computational efficiency (up to 60.9 GFLOP/s) and reduces energy consumption by up to 10.5×, demonstrating consistent gains across memory, latency, and energy dimensions.

Table 3. Ablation study on Code Llama-7B. “Quant. Only” applies 4-bit per-block weight quantization without the LLVM-based GEMV optimization. “BLAS Only” applies the GEMV-to-BLAS replacement on the full-precision model. “Full DITTO” combines both phases. Energy is estimated at 18 W sustained power draw.

Configuration	Pass@1	Pass@5	Pass@1	Pass@5	Memory (GB)	Tokens/s	Energy (J/tok)
	HumanEval+		MBPP+				
Full-precision	35.41	52.69	46.80	61.20	26.77	0.5	36.0
Quant. Only	36.66	50.14	47.29	56.75	4.2 (6.4×)	0.42 (0.8×)	42.9
BLAS Only	35.41	52.69	46.80	61.20	26.77	2.4 (4.8×)	7.5
Full DITTO	36.66	50.14	47.29	56.75	4.2 (6.4×)	3.5 (7.0×)	5.1 (7.1×)

4.4 RQ3: Ablation Study

To isolate the contributions of DITTO’s two optimization phases, we conduct an ablation study on Code Llama-7B using HumanEval+ and MBPP+. We evaluate three configurations: (1) *Quantization Only*, which applies 4-bit per-block weight quantization and the synthesized inference program without LLVM-based GEMV-to-BLAS replacement; (2) *BLAS Only*, which applies compilation-time GEMV optimization to the full-precision model without quantization; and (3) *Full DITTO*, which combines both phases. Table 3 summarizes the results. Because BLAS replacement does not modify the model weights, its accuracy matches that of the full-precision baseline. Similarly, Quantization Only and Full DITTO use identical quantized weights and therefore achieve the same accuracy.

The results in Table 3 show that the two phases address distinct bottlenecks and provide complementary benefits. Quantization achieves a 6.4× memory reduction, compressing the 7B model from 26.77 GB to 4.2 GB, which allows it to fit within the capacity of commodity laptops. However, it does not improve throughput, since performance slightly decreases to 0.8× of the baseline because of the runtime overhead of bit unpacking and centroid lookup during each GEMV computation. In contrast, BLAS replacement yields a 4.8× speedup by replacing naive loop-based GEMV with hardware-optimized library calls, but it provides no memory savings because the weights remain in full precision. When both phases are combined, DITTO achieves a 7.0× speedup, which is substantially higher than the 4.8× gain from BLAS replacement alone. This improvement arises because quantization reduces the memory bandwidth required for GEMV, which allows the BLAS kernels to operate more efficiently and to enable greater acceleration than either technique alone. The energy column in Table 3 further illustrates this effect. Quantization alone increases energy consumption (42.9 J/tok versus 36.0 J/tok for full precision), since the reduced memory footprint is offset by slower throughput caused by unpacking overhead. BLAS replacement alone reduces energy to 7.5 J/tok (4.8×) by directly improving computational throughput. When the two phases are combined, Full DITTO achieves 5.1 J/tok (7.1×), which exceeds the gains of either phase individually. These results confirm that quantization and compilation-time optimization are synergistic rather than independent, because quantization alleviates the memory-bandwidth bottleneck, BLAS replacement mitigates the compute bottleneck, and their integration yields amplified improvements in throughput, memory usage, and energy efficiency.

Answer to RQ3: DITTO’s two phases address complementary bottlenecks. Together, they achieve a 7.0× speedup, a 6.4× memory reduction, and a 7.1× reduction in energy consumption, outperforming either quantization or BLAS replacement alone due to their synergistic interaction.

Table 4. Comparison of quantization methods on Code Llama-7B (HumanEval+). AWQ-4bit stores weights in 4-bit format but dequantizes to int8 on CPU due to the absence of native 4-bit dot-product instructions.

Method	Bits	Pass@1	Mem. (GB)	Tok/s	Energy (J/tok)
Full-precision	32	35.41	26.8	0.5	36.0
int8	8	29.70 (-5.71)	6.7 (4.0×)	2.2 (4.4×)	8.2
AWQ-4bit	4	28.98 (-6.43)	5.4 (5.0×)	2.4 (4.8×)	7.5
DITTO	4	36.66 (+1.25)	4.2 (6.4×)	3.5 (7.0×)	5.1

5 Additional Analysis

5.1 Time Cost of Quantization and Compilation

We measure the quantization and compilation time required by DITTO to generate an optimized executable from a 7B-parameter Code LLM. On our evaluation machine (Apple M4 CPU, 24 GB RAM), the complete pipeline, which includes model quantization, bit-packing, and LLVM-based GEMV optimization, completes in approximately 17 minutes for all three models. This processing time is relatively short compared with the overall lifecycle of deploying a Code LLM, since it represents a one-time cost per model configuration. Once the optimized binary is generated, it can be reused indefinitely without recompilation, unless the model weights or target hardware change. Moreover, given the sustained performance improvements during inference, which reach up to 10.5× faster execution and 6.4× lower memory usage as demonstrated in RQ2, the time overhead is minimal and can be easily amortized over multiple inference runs.

5.2 Comparison with Other Quantization Methods

To compare DITTO with alternative quantization approaches, we evaluate it against Activation-aware Weight Quantization (AWQ) [39], a state-of-the-art method that maps individual weights to low-bit integers guided by activation statistics and has been shown to preserve accuracy at 4-bit precision for code LLMs [1], on Code Llama-7B under the same hardware environment. AWQ and similar approaches, such as GPTQ [18], require a calibration dataset to reduce quantization error. In contrast, DITTO performs clustering-based quantization directly on pretrained weights without calibration or retraining. Since curating a representative calibration dataset for code LLMs is non-trivial and beyond the scope of this work, we follow Afrin et al. [1] and use the publicly available AWQ 4-bit checkpoint from Hugging Face [31]. Inference is executed on CPU using the official Hugging Face pipeline without additional optimizations.

Table 4 shows that DITTO outperforms all baselines in accuracy, memory, throughput, and energy. AWQ achieves 2.4 tok/s, which is comparable to int8 (2.2 tok/s), but exhibits a larger pass@1 drop (-6.43% versus -5.71%). This comes from AWQ’s CPU execution path, since in the absence of native 4-bit dot-product instructions, 4-bit weights are dequantized to int8 or fp16 before computation [15]. Although storage uses 4 bits, arithmetic proceeds at higher precision, which limits throughput and energy efficiency. Consequently, AWQ consumes 7.5 J/tok, only slightly better than int8 (8.2 J/tok), whereas DITTO achieves 3.5 tok/s, a +1.25% pass@1 improvement, and 5.1 J/tok. This advantage arises from tightly coupling 4-bit per-block storage with native bit unpacking and BLAS-optimized GEMV execution, which eliminates intermediate dequantization and reduces both memory traffic and computational overhead.

More broadly, this comparison highlights a fundamental architectural distinction. GPU-oriented methods such as AWQ [39] and GPTQ [18] depend on specialized CUDA kernels for low-bit arithmetic and revert to higher-precision computation on CPUs, which diminishes their theoretical gains in CPU settings. Meanwhile, classical Vector Quantization (VQ) [52] and its variant Product

Table 5. Generalization of DITTO across model architectures, scales, and domains. Speedup and memory reduction are measured relative to full-precision inference.

Model	Arch.	Domain	Params	FP Mem. (GB)	DITTO Mem. (GB)	Speedup
CodeQwen-1.5-7B [50]	Qwen	Code	7B	27.1	4.4 (6.1×)	3.9×
Code Llama-13B [53]	LLaMA	Code	13B	53.0	9.7 (5.5×)	6.1×
TinyLLaMA [33]	LLaMA	General	110M	0.45	0.11 (4.1×)	4.2×

Quantization (PQ) [46] share a similar high-level idea with our method, i.e., reducing bit width through clustering. They quantize weight blocks into codebooks and perform inference through lookup-table accumulation. Although memory-efficient, these approaches introduce additional overhead from codebook access and reconstruction. Without optimizing the inference program, this overhead can offset the benefits of compression on CPUs, as shown in our RQ3 ablation study (Section 4.4). In contrast, our approach not only compresses weights but also optimizes the inference program. By replacing costly GEMV loops with BLAS-optimized kernels, DITTO compensates for the overhead of codebook lookups and enables efficient execution of low-bit models on commodity CPUs. This co-design of quantization and program-level optimization provides a technically distinct perspective on efficient LLM inference under tight resource constraints.

5.3 Generalization of DITTO

A natural question is whether the optimizations in DITTO are specific to the LLaMA architecture used in our primary evaluation. To assess generality, we apply DITTO to models with different architectures, scales, and domains (Table 5).

Architecture Generality. Both components of DITTO, namely weight quantization and GEMV-to-BLAS replacement, operate on standard Transformer linear layers and do not depend on architecture-specific mechanisms. We validate this on CodeQwen-1.5-7B [50], which adopts the Qwen architecture [6] and differs from LLaMA in its attention mechanism and feed-forward network design. Applying DITTO to CodeQwen-1.5-7B yields a 6.1× memory reduction, from 27.1 GB to 4.4 GB, and a 3.9× speedup, which is comparable to the results for Code Llama-7B. These results confirm that DITTO’s optimizations are not tied to a specific model architecture and can be effectively applied to other Transformer-based LLMs.

Scale and Domain Generality. We further evaluate DITTO on Code Llama-13B [53] and TinyLLaMA-110M [33] to examine scalability across model sizes and domains. For Code Llama-13B, memory usage decreases from 53.0 GB to 9.7 GB (5.5×) with a 6.1× speedup. Notably, the full-precision 13B model exceeds the 24 GB memory capacity of our evaluation machine, whereas the quantized version fits comfortably, which demonstrates that DITTO enables practical deployment at larger scales. For TinyLLaMA-110M, which is a general-purpose language model rather than a code-specialized one, memory usage decreases from 0.45 GB to 0.11 GB (4.1×) with a 4.2× speedup. This result indicates that DITTO’s optimizations are not limited to code-specific models and generalize to other domains as well. Taken together, these findings confirm that DITTO’s two-phase approach is broadly applicable across model scales and application domains.

Hardware Portability. Although our evaluation is conducted on an Apple M4 CPU, DITTO is not platform-specific. The quantization operates at the model level and produces a platform-independent compressed model. The GEMV-to-BLAS replacement is implemented as an LLVM IR pass that emits calls to the standard CBLAS interface. At link time, these calls bind to the local BLAS provider, such as Apple Accelerate, OpenBLAS, or Intel MKL. Since optimized BLAS libraries are available for modern CPU architectures including x86, ARM, and RISC-V, the same compiled pipeline can

be deployed by relinking against the target platform's BLAS implementation. Memory reduction remains consistent across platforms, whereas throughput improvement depends on the quality of the underlying BLAS library.

5.4 Implications for Software Engineering

Takeaway 1 (for software developers): With DITTO, developers can run Code LLMs locally on standard laptops without GPUs, achieving latency below 300 ms with approximately 4 GB of memory. This enables fast, privacy-preserving code completion and analysis directly within IDEs, without transmitting proprietary code to cloud services.

Takeaway 2 (for engineering teams): Teams can integrate DITTO as a build-time step that converts pretrained models into compact, GPU-free C executables in approximately 17 minutes (Section 5.1). The resulting binary depends only on a system BLAS library, which facilitates seamless integration into CI/CD pipelines and supports on-device deployment of AI coding assistants.

Takeaway 3 (for SE researchers): DITTO illustrates that co-designing model compression with program-level optimization yields synergistic gains that exceed those of either technique alone (Section 4.4). This principle can guide future research on efficient LLM-powered software engineering tools, such as on-device program repair and privacy-sensitive code search.

5.5 Threats to Validity

A potential threat to internal validity arises from randomness in code generation and quantization. To mitigate this issue, we repeated each experiment five times with different random seeds and reported the average results, following Arcuri and Briand [4]. Although clustering in quantization depends on centroid initialization, our iterative refinement process converges to a stable configuration, which reduces sensitivity to randomness. Another concern is the correctness of our custom LLVM pass, which we addressed through extensive unit testing and by verifying functional equivalence between the original and optimized code on representative examples. Regarding external validity, we evaluated three widely used code-specialized LLMs on two standard benchmarks using an Apple M4 CPU. These models, datasets, and hardware are representative of common usage scenarios for Code LLMs. Nevertheless, future work could further examine generalizability across additional models, benchmarks, and platforms. For construct validity, our metrics, including pass@1, pass@5, inference speed, and memory usage, are standard in LLM evaluation and directly reflect practical usability.

6 Related Work

The first line of related work focuses on model compression [43, 56], which reduces the size of LLMs to improve inference latency, memory usage, and energy consumption. As an early effort on compressing Code LLMs, Shi et al. [57] use a genetic algorithm to simplify model architectures and apply knowledge distillation to shrink models, which achieves an average 4.23× latency improvement. They subsequently introduce Avatar [55], which jointly optimizes model size, latency, energy consumption, and carbon footprint. Similarly, Su and McMillan [58] distill GPT-3.5 into a smaller model for code summarization. Other approaches include quantization [66] and low-rank decomposition [34], both of which achieve notable efficiency gains for code generation. Recent work further examines the non-functional properties of compressed Code LLMs, including robustness [17], privacy [22], and output code quality [1].

There are also efforts on software-hardware co-design for efficient LLM inference [48, 49, 65, 69], which jointly optimize the model and the underlying hardware architecture to improve performance. Some studies target high-performance platforms such as GPUs or ASIC accelerators, including

VQ-LLM [41], MVQ [38], and VQT-CiM [26]. VQ-LLM depends on custom CUDA kernels, MVQ relies on GPU tensor cores, and VQT-CiM targets ferroelectric compute-in-memory accelerators. These hardware-specific implementations differ from our focus on commodity CPU-only devices without such hardware support. In addition, neural codebook methods such as QINCo2 [62] address a fundamentally different problem, which is compressing input vectors into quantized embeddings for similarity search rather than compressing model weights for LLM inference. Such methods also use neural networks to construct codebooks and reconstruct vectors, whereas DITTO performs table lookup to recover weights and co-designs quantization with compilation-time GEMV-to-BLAS replacement. This integration enables low-bitwidth execution while also systematically optimizing the inference program itself, which has not been explored in prior work. We therefore believe that our method is both technically novel and provides a unique perspective in this field.

Program-level optimization of Code LLMs has also been explored, primarily by restructuring inputs or reducing token counts to lower processing costs, since inference time scales with input length. Token pruning methods remove less important tokens to shorten sequences [23, 59, 73]. For example, Zhang et al. [73] propose DietCode, which selects high-attention statements and tokens to reduce the computational cost of Code LLMs. Although effective at shrinking inputs, these approaches do not optimize the underlying inference program itself, which limits overall system-level gains. In contrast, our work directly improves the low-level execution of the inference pipeline and complements input-level reductions with compiler-based acceleration.

Beyond efforts specific to Code LLMs, compiler research has long leveraged pattern matching and LLVM IR passes for program optimization, such as LoopTactics [8], KernelFaRer [13], and MMTR [27], which perform nested loop and matrix multiplication optimizations through LLVM-based idiom recognition and rewriting. However, these systems target general-purpose program optimization rather than LLM inference workloads. Our work instead introduces a specialized framework tailored to Code LLM inference, which provides a focused and high-performance solution for this emerging domain.

7 Conclusion and Future Work

In this work, we present DITTO, a synthesis- and compiler-inspired framework for optimizing Code LLM inference programs. By combining K-Means-based weight quantization with an LLVM-based GEMV rewrite, DITTO generates low-bitwidth, memory- and compute-efficient executables that run on commodity CPUs. When evaluated on Code Llama, MagicCoder, and OpenCodeInterpreter, DITTO achieves up to a $10.5\times$ speedup and a $6.4\times$ memory reduction over the original pipelines while maintaining near full-precision accuracy, with an average pass@1 loss of only 0.27%. Compared with an INT8 baseline, it delivers up to 6.96% higher pass@1, a $2.2\times$ speedup, and $1.6\times$ lower memory usage. The end-to-end quantization and compilation process takes only 17 minutes for a 7B model, which makes deployment practical on personal devices. Currently, DITTO targets inference pipelines based on the llama.c design, because of its popularity and lightweight, dependency-free structure, which enables fine-grained control over model loading and kernel replacement. In future work, we plan to extend support to other implementations, such as llama.cpp and PyTorch or TensorFlow pipelines, and to broaden the optimization pass to additional kernels so that it can enable batch processing and support encoder-decoder and multimodal architectures.

Data Availability

The code of DITTO is available at <https://github.com/jiekeshi/ditto>.

Acknowledgments

This research / project is supported by the A*STAR under its 2nd CSIRO and A*STAR: Research-Industry (2+2) Partnership Program (Award R24I5IR047). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the A*STAR. We would also like to thank the anonymous reviewers for their valuable feedback and suggestions.

References

- [1] Saima Afrin, Bowen Xu, and Antonio Mastropaolo. 2025. Is Quantization a Deal-Breaker? Empirical Insights From Large Code Models. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 1–13. doi:10.1109/ICSME64153.2025.00049
- [2] Toufique Ahmed, Kunal Suresh Pai, Premkumar Devanbu, and Earl Barr. 2024. Automatic Semantic Augmentation of Language Model Prompts (for Code Summarization). In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 220, 13 pages. doi:10.1145/3597503.3639183
- [3] Apple. 2025. Apple's Accelerate framework. <https://developer.apple.com/documentation/accelerate/blas/>.
- [4] Andrea Arcuri and Lionel Briand. 2011. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering (Waikiki, Honolulu, HI, USA) (ICSE '11)*. Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/1985793.1985795
- [5] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021). doi:10.48550/arXiv.2108.07732
- [6] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, and et al. 2023. Qwen Technical Report. doi:10.48550/arXiv.2309.16609 arXiv:2309.16609 [cs.CL]
- [7] L Susan Blackford, Antoine Petit, Roldan Pozo, Karin Remington, R Clint Whaley, James Demmel, Jack Dongarra, Iain Duff, Sven Hammarling, Greg Henry, et al. 2002. An updated set of basic linear algebra subprograms (BLAS). *ACM Trans. Math. Software* 28, 2 (2002), 135–151. doi:10.1145/567806.567807
- [8] Lorenzo Chelini, Oleksandr Zinenko, Tobias Grosser, and Henk Corporaal. 2019. Declarative Loop Tactics for Domain-specific Optimization. *ACM Trans. Archit. Code Optim.* 16, 4, Article 55 (Dec. 2019), 25 pages. doi:10.1145/3372266
- [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374
- [10] Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. 2020. ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. (2020). <https://openreview.net/forum?id=r1xMH1BtvB>
- [11] Cursor. 2025. Cursor - The AI Code Editor. <https://www.cursor.com>.
- [12] Marek Czachor and Jan Naudts. 2007. Regularization as quantization in reducible representations of CCR. *International Journal of Theoretical Physics* 46, 1 (2007), 70–101. doi:10.1007/s10773-006-9199-8
- [13] João P. L. De Carvalho, Braedy Kuzma, Ivan Korostelev, José Nelson Amaral, Christopher Barton, José Moreira, and Guido Araújo. 2021. KernelFaRer: Replacing Native-Code Idioms with High-Performance Library Calls. *ACM Trans. Archit. Code Optim.* 18, 3, Article 38 (June 2021), 22 pages. doi:10.1145/3459010
- [14] Hugging Face. 2025. Hugging Face – The AI community building the future. <https://huggingface.co>.
- [15] Hugging Face. 2026. GPTQ. <https://huggingface.co/docs/transformers/en/quantization/gptq>.
- [16] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. 31–53. doi:10.1109/icse-fose59343.2023.00008
- [17] Sen Fang, Weiyan Ding, Antonio Mastropaolo, and Bowen Xu. 2025. Smaller = Weaker? Benchmarking Robustness of Quantized LLMs in Code Generation. doi:10.48550/arXiv.2506.22776 arXiv:2506.22776 [cs.SE]
- [18] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2023. OPTQ: Accurate Quantization for Generative Pre-trained Transformers. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=tcbBPnfwxS>
- [19] Prakhhar Ganesh, Yao Chen, Xin Lou, Mohammad Ali Khan, Yin Yang, Hassan Sajjad, Preslav Nakov, Deming Chen, and Marianne Winslett. 2021. Compressing Large-Scale Transformer-Based Models: A Case Study on BERT. *Transactions of the Association for Computational Linguistics* 9 (2021), 1061–1080. doi:10.1162/tacl_a_00413
- [20] Georgi Gerganov. 2023. ggerganov/llama.cpp: LLM inference in C/C++. <https://github.com/ggerganov/llama.cpp>.
- [21] GitHub. 2025. GitHub Copilot · Your AI pair programmer. <https://github.com/features/copilot/>.

- [22] Md Nazmul Haque, Hua Yang, Zhou Yang, and Bowen Xu. 2025. How Quantization Impacts Privacy Risk on LLMs for Code? [doi:10.48550/arXiv.2508.00128](https://doi.org/10.48550/arXiv.2508.00128) arXiv:2508.00128 [cs.SE]
- [23] Dávid Hidvégi, Khashayar Etemadi, Sofia Bobadilla, and Martin Monperrus. 2024. CigaR: Cost-efficient Program Repair with LLMs. [doi:10.48550/arXiv.2402.06598](https://doi.org/10.48550/arXiv.2402.06598) arXiv:2402.06598 [cs.SE]
- [24] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* (2015). [doi:10.48550/arXiv.1503.02531](https://doi.org/10.48550/arXiv.1503.02531)
- [25] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8, Article 220 (Dec. 2024), 79 pages. [doi:10.1145/3695988](https://doi.org/10.1145/3695988)
- [26] Xuchu Huang, Haonan Du, Min Zhou, Zheyu Yan, Cheng Zhuo, and Xunzhao Yin. 2025. VQT-CiM: Accelerating vector quantization enhanced transformer with ferroelectric compute-in-memory. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7. [doi:10.1109/dac63849.2025.11133264](https://doi.org/10.1109/dac63849.2025.11133264)
- [27] Benedikt Huber and Andreas Krall. 2025. Pattern Matching, Transformation and Code Replacement on a Polyhedral Representation of Nested Loops. In *Proceedings of the 22nd ACM International Conference on Computing Frontiers (CF '25)*. Association for Computing Machinery, New York, NY, USA, 176–184. [doi:10.1145/3719276.3725197](https://doi.org/10.1145/3719276.3725197)
- [28] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A Survey on Large Language Models for Code Generation. *ACM Trans. Softw. Eng. Methodol.* 35, 2, Article 58 (Jan. 2026), 72 pages. [doi:10.1145/3747588](https://doi.org/10.1145/3747588)
- [29] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1430–1442. [doi:10.1109/icse48619.2023.00125](https://doi.org/10.1109/icse48619.2023.00125)
- [30] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. InferFix: End-to-End Program Repair with LLMs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (San Francisco, CA, USA) (ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1646–1656. [doi:10.1145/3611643.3613892](https://doi.org/10.1145/3611643.3613892)
- [31] Tom Jobbins. 2026. TheBloke (Tom Jobbins). <https://huggingface.co/TheBloke>.
- [32] Andrej karpathy. 2025. karpathy/llama2.c: Inference Llama 2 in one file of pure C. <https://github.com/karpathy/llama2.c.git>.
- [33] Andrej Karpathy. 2026. karpathy/tinylamas · Hugging Face. <https://huggingface.co/karpathy/tinylamas>.
- [34] Ayush Kaushal, Tejas Vaidhya, and Irina Rish. 2023. LORD: Low Rank Decomposition Of Monolingual Code LLMs For One-Shot Compression. [doi:10.48550/arXiv.2309.14021](https://doi.org/10.48550/arXiv.2309.14021) arXiv:2309.14021 [cs.CL]
- [35] Toufik Kechaoui, Mohamed Wassim Ouhab, Badis Djamaa, and Mustapha Reda Senouci. 2025. Locally-deployed Open-source LLMs for Code Generation: Promises and Challenges. In *2025 7th International Conference on Pattern Analysis and Intelligent Systems (PAIS)*. 1–6. [doi:10.1109/PAIS66004.2025.11126523](https://doi.org/10.1109/PAIS66004.2025.11126523)
- [36] Seongho Kim, Jihyun Moon, Juntaek Oh, Insu Choi, and Joon-Sung Yang. 2025. Survey and Evaluation of Converging Architecture in LLMs based on Footsteps of Operations. *IEEE Open Journal of the Computer Society* (2025). [doi:10.1109/ojcs.2025.3587005](https://doi.org/10.1109/ojcs.2025.3587005)
- [37] Jiliang Li, Yifan Zhang, Zachary Karas, Collin McMillan, Kevin Leach, and Yu Huang. 2024. Do Machines and Humans Focus on Similar Code? Exploring Explainability of Large Language Models in Code Summarization. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (Lisbon, Portugal) (ICPC '24)*. Association for Computing Machinery, New York, NY, USA, 47–51. [doi:10.1145/3643916.3644434](https://doi.org/10.1145/3643916.3644434)
- [38] Shuaiting Li, Chengxuan Wang, Juncan Deng, Zeyu Wang, Zewen Ye, Zongsheng Wang, Haibin Shen, and Kejie Huang. 2025. Mvq: Towards efficient dnn compression and acceleration with masked vector quantization. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 731–745. [doi:10.1145/3669940.3707268](https://doi.org/10.1145/3669940.3707268)
- [39] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. 87–100. [doi:10.1145/3714983.3714987](https://doi.org/10.1145/3714983.3714987)
- [40] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Thirty-seventh Conference on Neural Information Processing Systems*. [doi:10.52202/075280-0943](https://doi.org/10.52202/075280-0943)
- [41] Zihan Liu, Xinhao Luo, Junxian Guo, Wentao Ni, Yangjie Zhou, Yue Guan, Cong Guo, Weihao Cui, Yu Feng, Minyi Guo, Yuhao Zhu, Minjia Zhang, Chen Jin, and Jingwen Leng. 2025. VQ-LLM: High-performance Code Generation for Vector Quantization Augmented LLM Inference. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1496–1509. [doi:10.1109/HPCA61900.2025.00112](https://doi.org/10.1109/HPCA61900.2025.00112)
- [42] S. Lloyd. 2006. Least squares quantization in PCM. *IEEE Trans. Inf. Theor.* 28, 2 (Sept. 2006), 129–137. [doi:10.1109/TIT.1982.1056489](https://doi.org/10.1109/TIT.1982.1056489)

- [43] David Lo. 2023. Trustworthy and Synergistic Artificial Intelligence for Software Engineering: Vision and Roadmaps. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. 69–85. doi:10.1109/icse-fose59343.2023.00010
- [44] Yunbo Lyu, Zhou Yang, Jieke Shi, Jianming Chang, Yue Liu, and David Lo. 2025. "My productivity is boosted, but ..." Demystifying Users' Perception on AI Coding Assistants. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE '25)*. Association for Computing Machinery, New York, NY, USA. doi:10.1109/ase63991.2025.00024
- [45] Henry B Mann and Donald R Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* (1947), 50–60. doi:10.1214/aoms/1177730491
- [46] Yusuke Matsui, Yusuke Uchida, Hervé Jégou, and Shin'ichi Satoh. 2018. A survey of product quantization. *ITE Transactions on Media Technology and Applications* 6, 1 (2018), 2–10. doi:10.3169/mta.6.2
- [47] Bonan Min, Hayley Ross, Elior Sulem, Amir Poursan Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. 2023. Recent Advances in Natural Language Processing via Large Pre-trained Language Models: A Survey. *ACM Comput. Surv.* 56, 2, Article 30 (Sept. 2023), 40 pages. doi:10.1145/3605943
- [48] Zhiwen Mo, Lei Wang, Jianyu Wei, Zhichen Zeng, Shijie Cao, Lingxiao Ma, Naifeng Jing, Ting Cao, Jilong Xue, Fan Yang, et al. 2025. LUT Tensor Core: A Software-Hardware Co-Design for LUT-Based Low-Bit LLM Inference. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 514–528. doi:10.1145/3695053.3731057
- [49] Gunho Park, Baeseong park, Minsub Kim, Sungjae Lee, Jeonghoon Kim, Beomseok Kwon, Se Jung Kwon, Byeongwook Kim, Youngjoo Lee, and Dongsoo Lee. 2024. LUT-GEMM: Quantized Matrix Multiplication based on LUTs for Efficient Inference in Large-Scale Generative Language Models. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=gLARhFLE0F>
- [50] Qwen. 2026. Qwen/CodeQwen1.5-7B. <https://huggingface.co/Qwen/CodeQwen1.5-7B>.
- [51] Shanto Rahman, Abdelrahman Baz, Sasa Misailovic, and August Shi. 2024. Quantizing Large-Language Models for Predicting Flaky Tests . In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE Computer Society, Los Alamitos, CA, USA, 93–104. doi:10.1109/icst60714.2024.00018
- [52] Babak Rokh, Ali Azarpeyvand, and Alireza Khanteymoori. 2023. A comprehensive survey on model quantization for deep neural networks in image classification. *ACM Transactions on Intelligent Systems and Technology* 14, 6 (2023), 1–50. doi:10.1145/3623402
- [53] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. doi:10.48550/arXiv.2308.12950 arXiv:2308.12950 [cs.CL]
- [54] Agnia Sergeyuk, Yaroslav Golubev, Timofey Bryksin, and Iftekhar Ahmed. 2025. Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward. *Information and Software Technology* 178 (2025), 107610. doi:10.1016/j.infsof.2024.107610
- [55] Jieke Shi, Zhou Yang, Hong Jin Kang, Bowen Xu, Junda He, and David Lo. 2024. Greening Large Language Models of Code. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society (Lisbon, Portugal) (ICSE-SEIS'24)*. Association for Computing Machinery, New York, NY, USA, 142–153. doi:10.1145/3639475.3640097
- [56] Jieke Shi, Zhou Yang, and David Lo. 2025. Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 137 (May 2025), 22 pages. doi:10.1145/3708525
- [57] Jieke Shi, Zhou Yang, Bowen Xu, Hong Jin Kang, and David Lo. 2023. Compressing Pre-trained Models of Code into 3 MB. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (Rochester, MI, USA) (ASE '22)*. Association for Computing Machinery, New York, NY, USA, Article 24, 12 pages. doi:10.1145/3551349.3556964
- [58] Chia-Yi Su and Collin McMillan. 2024. Distilled GPT for source code summarization. *Automated Software Engineering* 31, 1 (2024), 22. doi:10.1007/s10515-024-00421-4
- [59] Zhensu Sun, Xiaoning Du, Zhou Yang, Li Li, and David Lo. 2024. AI Coders Are among Us: Rethinking Programming Language Grammar towards Efficient Code Generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (Vienna, Austria) (ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1124–1136. doi:10.1145/3650212.3680347
- [60] Chandra Thapa, Seung Ick Jang, Muhammad Ejaz Ahmed, Seyit Camtepe, Josef Pieprzyk, and Surya Nepal. 2022. Transformer-Based Language Models for Software Vulnerability Detection. In *Proceedings of the 38th Annual Computer Security Applications Conference (Austin, TX, USA) (ACSAC '22)*. Association for Computing Machinery, New York, NY, USA, 481–496. doi:10.1145/3564625.3567985

- [61] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, and et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. doi:10.48550/arXiv.2307.09288 arXiv:2307.09288 [cs.CL]
- [62] Théophane Vallaes, Matthew J. Muckley, Jakob Verbeek, and Matthijs Douze. 2025. Qinc2: Vector Compression and Search with Improved Implicit Neural Codebooks. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=2zMHZ569S>
- [63] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [64] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936. doi:10.1109/tse.2024.3368208
- [65] Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. 2025. T-mac: Cpu renaissance via table lookup for low-bit llm deployment on edge. In *Proceedings of the Twentieth European Conference on Computer Systems*. 278–292. doi:10.1145/3689031.3696099
- [66] Xiaokai Wei, Sujun Kumar Gonugondla, Shiqi Wang, Wasi Ahmad, Baishakhi Ray, Haifeng Qian, Xiaopeng Li, Varun Kumar, Zijian Wang, Yuchen Tian, Qing Sun, Ben Athiwaratkun, Mingyue Shang, Murali Krishna Ramanathan, Parminder Bhatia, and Bing Xiang. 2023. Towards Greener Yet Powerful Code Generation via Quantization: An Empirical Study. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (San Francisco, CA, USA) (ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 224–236. doi:10.1145/3611643.3616302
- [67] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2024. Magicoder: Empowering Code Generation with OSS-Instruct. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 52632–52657. <https://proceedings.mlr.press/v235/wei24h.html>
- [68] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated Program Repair in the Era of Large Pre-trained Language Models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1482–1494. doi:10.1109/icse48619.2023.00129
- [69] Ceyu Xu, Yongji Wu, Xinyu Yang, Beidi Chen, Matthew Lentz, Danyang Zhuo, and Lisa Wu Wills. 2025. LLM.265: Video Codecs are Secretly Tensor Codecs. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture, MICRO 2025, Seoul, Republic of Korea, October 18–22, 2025*. ACM, 445–460. doi:10.1145/3725843.3756078
- [70] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming (San Diego, CA, USA) (MAPS 2022)*. Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/3520312.3534862
- [71] Chao Zeng, Songwei Liu, Yusheng Xie, Hong Liu, Xiaojian Wang, Miao Wei, Shu Yang, Fangmin Chen, and Xing Mei. 2025. Abq-llm: Arbitrary-bit quantized inference acceleration for large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 22299–22307. doi:10.1609/aaai.v39i21.34385
- [72] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Shengcheng Yu, Weisong Sun, Yun Yang, and Zhenyu Chen. 2026. A survey on large language models for software engineering. *Sci. China Inf. Sci.* 69, 4 (2026). doi:10.1007/S11432-025-4670-0
- [73] Zhaowei Zhang, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2022. Diet Code is Healthy: Simplifying Programs for Pre-Trained Models of Code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Singapore, Singapore) (ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 1073–1084. doi:10.1145/3540250.3549094
- [74] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Yiwen Hu, Jian-Yun Nie, and Ji-Rong Wen. 2026. A Survey of Large Language Models. *Frontiers Comput. Sci.* 20, 12 (2026), 2012627. doi:10.1007/S11704-026-60308-3
- [75] Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhui Chen, and Xiang Yue. 2024. OpenCodeInterpreter: Integrating Code Generation with Execution and Refinement. In *Findings of the Association for Computational Linguistics ACL 2024*. 12834–12859. doi:10.18653/v1/2024.findings-acl.762
- [76] Zibin Zheng, Kaiwen Ning, Yanlin Wang, Jingwen Zhang, Dewu Zheng, Mingxi Ye, and Jiachi Chen. 2024. A Survey of Large Language Models for Code: Evolution, Benchmarking, and Future Trends. doi:10.48550/arXiv.2311.10372 arXiv:2311.10372 [cs.SE]
- [77] Xin Zhou, Ting Zhang, and David Lo. 2024. Large Language Model for Vulnerability Detection: Emerging Results and Future Directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results (Lisbon, Portugal) (ICSE-NIER'24)*. Association for Computing Machinery, New York, NY,

USA, 47–51. doi:[10.1145/3639476.3639762](https://doi.org/10.1145/3639476.3639762)

- [78] Terry Yue Zhuo, Vu Minh Chien, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen GONG, James Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, David Lo, Binyuan Hui, Niklas Muennighoff, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *The Thirteenth International Conference on Learning Representations*. doi:[10.48550/arXiv.2406.15877](https://doi.org/10.48550/arXiv.2406.15877)